

Magic2

Deep RNA-Sequencing Analyser

User Guide

Jean Thierry-Mieg, Danielle Thierry-Mieg, Greg Boratyn

National Library of Medicine, National Institutes of Health
8600 Rockville Pike, Bethesda MD 20894, USA

`mieg@ncbi.nlm.nih.gov boratyng@ncbi.nlm.nih.gov`

Source code: <https://github.com/NLM-DIR/Magic2>

Contents

1	Overview	2
2	System Requirements	2
3	Installation	2
3.1	Using pre-built executables	2
3.2	Building from source	3
4	Quick Start	3
4.1	Example 1: Align reads against a raw genome	3
4.2	Example 2: Align reads against an annotated genome	3
4.3	Example 3: Stream and align SRA data	4
5	Input Data	4
5.1	Single-file input (-i)	4
5.2	Run manifest (-I)	5
5.2.1	Format	5
5.2.2	Run naming rules	5
5.2.3	Column 3 is optional	5
5.2.4	Example	6
5.2.5	Parallelism across runs	6
5.3	Streaming SRA runs on the fly	7
5.4	Downloading SRA runs	7
5.5	Streaming and caching simultaneously	8
5.6	Processing multiple runs in parallel	8
5.7	Processing multiple sets runs in parallel on a multicore machine	8
6	Building a Target Index	9
6.1	Overview	9
6.2	The target configuration file (tConfig)	9
6.2.1	Annotation file formats	10
6.3	Creating the index	10
7	Running an Alignment	11
7.1	Basic usage	11
7.2	Output format options	11
7.3	Requesting the full output suite	12
8	Output Files	12
8.1	Alignment files	12
8.2	Intron support table	12
8.3	Poly-A addition sites	13
8.4	Gene expression counts	13
8.5	Coverage plots	13
8.6	Transcription start and end sites, repetitions, breakpoints	14
8.7	Quality-control metrics	14

9	Command-Line Reference	15
9.1	Index construction	15
9.2	Input	16
9.3	Output	16
10	Worked Examples	17
10.1	Quality survey of SRA runs without storing data	17
10.2	Full analysis of a paired-end Illumina run	17
10.3	Parallel processing of multiple SRA runs	18
10.4	Merging multi-run output tables	18
11	Troubleshooting	18

1 Overview

Magic2 is an accurate, sensitive, RNA deep-sequencing analyzer designed for both short reads (Illumina paired-end) and long reads (PacBio, Nanopore, Roche). In a single pass it produces alignment files, gene expression counts, intron support tables, poly-A addition sites, coverage plots, transcript-ends candidates and quality-control metrics.

Key capabilities:

- Aligns reads against a raw genome or an annotated genome (GTF/GFF).
- Reads local FASTA/FASTQ files (gzip-compressed or plain) or streams data on the fly from the NCBI Short Read Archive (SRA).
- Automatically recognises adaptor sequences, sequencing technology, and read length—no strategy parameters required.
- Handles paired-end reads presented in separate files, or preferably in a single interleaved file.
- Exports alignments in SAM, BAM, Magic `.hits`, or tabular format.
- Adapts its parallelism to the available hardware— no thread count parameter needed
- May offload on request part of the analysis to a GPU when one is present, although the performance gains are so far marginal.

Note: Magic2 runs a dry run on the first 10 Megabases to detect read characteristics before committing to a full alignment strategy. You do not need to specify whether reads are DNA or RNA, short or long, or whether they contain adaptors.

2 System Requirements

Component	Minimum / Recommended
Operating system	Linux (x86-64) or macOS
RAM	32 GB minimum; 64+ GB recommended for human genome
CPU cores	4 minimum; performance scales up with core count
Disk	Varies with genome size; ~22 GB for the human index
Network	Fast connection to NCBI recommended for SRA streaming
GPU (optional)	Any CUDA-capable GPU, may marginally accelerate the analysis

3 Installation

3.1 Using pre-built executables

Pre-compiled binaries for Linux and macOS are available from:

```
https://github.com/NLM-DIR/Magic2/releases
```

Download the archive for your platform, extract it, and add the `bin/` directory to your `PATH`, or copy the `magic2` executable to `/usr/local/bin`.

To verify the installation:

```
magic2 --help
```

3.2 Building from source

The source code is available on GitHub under an open-source licence.

```
git clone git@github.com:NLM-DIR/Magic2
```

Then compile:

```
tcsh
cd Magic2
setenv ACEDB_MACHINE LINUX_4_OPT or MAC_X_64_OPT
make libs
cd wsa
make
```

The executable is created at Magic2/bin.LINUX_4_OPT/magic2. Add this directory to your PATH:

```
export PATH=$PATH:/path/to/Magic2/bin.LINUX_4_OPT
```

Note: Locally compiled binaries may outperform the pre-built releases because the compiler can target the specific instruction sets of your hardware.

4 Quick Start

This section shows three minimal end-to-end examples to get you started immediately. Full details for each step are in the sections that follow.

4.1 Example 1: Align reads against a raw genome

Two steps:

```
# Step 1: Build the index using the genome fasta file (one time only)
magic2 --createIndex IDX.genome -t genome.fasta.gz

# Step 2: Align the reads
magic2 -x IDX.genome -i reads.fasta.gz -o results/
```

genome.fasta.gz must contain all chromosomes in a single FASTA file. Results appear in the results/ directory.

4.2 Example 2: Align reads against an annotated genome

Three steps (see the details in Section 6.1):

```
# Step 1: Create a target configuration file (tConfig)
```

```
# Format: 2 tab-separated columns per line: type/filename
# G = genomic sequence    R = ribosomal RNA
# M = mitochondria        A = annotation (GTF/GFF)
cat > human.tConfig << 'EOF'
G   T2T.chromosomes.fasta.gz
R   T2T.rRNA.fasta.gz
M   T2T.mitochondria.fasta.gz
A   T2T.annotation.gtf.gz
EOF

# Step 2: Build the annotated index
magic2 --createIndex IDX.T2T -T human.tConfig

# Step 3: Align and produce the full output suite
magic2 -x IDX.T2T -i reads.fastq.gz -o results/ --full
```

4.3 Example 3: Stream and align SRA data

```
# Align three SRA runs on the fly, no local download required
magic2 -x IDX.T2T --srr SRR1234,SRR1235,SRR1236 -o results/ --full
```

5 Input Data

5.1 Single-file input (-i)

For a single sequencing file or paired-end pair, use `-i`. This option accepts exactly one file or one paired-end pair. To process multiple files or samples, use `-I` (Section 5.2). The file format is detected automatically from the extension (`.fasta`, `.fastq`, `.fastc`, `.gz`).

Single-end reads:

```
magic2 -x IDX -i reads.fastq.gz -o out/
```

Paired-end reads in two separate files (joined with +):

```
magic2 -x IDX -i reads_R1.fastq.gz+reads_R2.fastq.gz -o out/
```

Paired-end reads in interleaved format:

```
magic2 -x IDX -i sample.sample_12.fastq.gz -o out/
```

In interleaved format, each pair occupies consecutive records with identifiers ending in `/1` and `/2`: four lines per pair in FASTA (two records of header + sequence), eight lines per pair in FASTQ (two records of header + sequence + + + quality).

To be automatically recognized, the file should be called `*.sample_12.fast[aq](.gz)`. If not, use `-i interleaved` on the command line, or write `interleaved` in the third column of the read manifest

Reading from a pipe:

```
zcat *.fastq.gz | magic2 -x IDX -i - --fastq --run runName -o out/
```

Note: Run label and adaptor sequences may be supplied as command-line flags (`--adaptor1`, `--adaptor2`, `--run`). These user provided flags override the automatic recognition system. When `-I` is used (see below), values given on the command line can themselves be overridden run by run using optional column 3 of the run manifest file.

5.2 Run manifest (-I)

When a project involves multiple sequencing files or samples, describe them in a plain-text run manifest file and pass it with `-I`. This is the recommended approach for any analysis beyond a single test run.

`-i` and `-I` are mutually exclusive, mirroring the `-t` / `-T` distinction for index construction.

5.2.1 Format

The manifest is a tab-separated text file with up to three columns and one row per input file. Lines beginning with `#` are ignored.

Column	Content	Notes
1	Filename	SRR identifier or path to a FASTA or FASTQ file, plain or gzip-compressed. For paired-end reads in two files, join the two paths with <code>+</code> , e.g. <code>R1.fastq.gz+R2.fastq.gz</code> .
2	Run name	A nickname describing the sample. Multiple rows sharing the same run name are merged into a single output.
3	Properties	Optional comma-separated list of descriptors (defined below).

5.2.2 Run naming rules

The second column, the run name, allows to choose a meaningful sample name and to associate to one or several data files to the same sample. This is useful for example to group technical replicas (as opposed to biological replicas). The program will export a single output table (alignments, gene counts, intron counts, coverage plots...) per run. By default, if column 2 is empty, the SRR identifier or the file name stripped of its path information and suffix will be used as a run name. (file=`a/b/c.fasta.gz` is reduced to run=`c`).

Run names (column 2) must consist of letters, digits, and underscores only (`[A-Za-z0-9_]`). Spaces, slashes, and special characters would cause errors and will be rejected, because run names are used directly as components of output file and directory names.

5.2.3 Column 3 is optional

The third column is optional as the properties can in general be auto-detected. By default, the file format is implied by the file suffix: `.fasta`, `.fastq` (or `.fastc` for backward compatibility with

MagicBlast). The files called `.sample_12.fast[aq]` are interleaved (consecutive records ending in `/1` and `/2`). The nature of the data: DNA or RNA, and the eventual adaptors are recognized automatically by the code by performing a dry run on the first 10 Megabases. Thus, the third column is only needed to impose a choice and override the auto-detection. The recognized keyword are:

Keyword	Effect
RNA	Align in RNA mode (intron-aware, poly-A detection).
DNA	Align in DNA mode (continuous alignments only).
fasta	Treat the file as FASTA.
fastq	Treat the file as FASTQ.
fastc	Treat the file as FASTC.
interleaved	Treat the file as interleaved paired-end.
Adaptor1=seq	Read-1 exit adaptor sequence.
Adaptor2=seq	Read-2 exit adaptor sequence.

5.2.4 Example

Here is an example with 5 files and 3 run names:

```
# my_project.rConfig
# Col 1: file(s)           Col 2: run           Col 3: properties
lib1_R1.fastq.gz+lib1_R2.fastq.gz  sample_A
lib2.fasta.gz                    sample_A
lib3_R1.fastq.gz+lib3_R2.fastq.gz  sample_B           RNA
lib4.fastq.gz                    sample_C           DNA,Adaptor1=AGATCGGAAG
```

In this example, `lib1` and `lib2` are technical sub-libraries for the same biological sample. Because they share the run name `sample_A`, Magic2 aligns both files and merges their output into a single BAM, gene count table, and coverage track for `sample_A`, whereas `sample_B` and `sample_C` are independent. The adaptors will be auto-detected in samples A and B, imposed in C. RNA/DNA will be auto-detected in A, imposed in B and C. All three runs are processed simultaneously.

A manifest with only a filename column is also valid. Magic2 will infer the format from the file suffix and auto-detect all other properties:

```
sample_A_R1.fastq.gz+sample_A_R2.fastq.gz
sample_B_R1.fastq.gz+sample_B_R2.fastq.gz
sample_C.fasta.gz
```

In this minimal case each file is treated as its own run; the run name defaults to the filename without path or extension: `sample_A`, `sample_B` and `sample_C`.

5.2.5 Parallelism across runs

All runs described in the manifest are processed concurrently. Magic2 mixes data packets from different runs through the same pipeline, fully utilising available hardware regardless of how reads are distributed across files. There is no practical limit on the total number of reads: the pipeline's

flow-control mechanism prevents RAM exhaustion by regulating the rate at which new data blocks enter the pipeline.

5.3 Streaming SRA runs on the fly

If you have a fast connection to NCBI, you can align without storing the raw reads locally. Use `--srr` instead of `-i`:

```
magic2 -x IDX --srr SRR1,SRR2,SRR3 -o out/
```

Streaming from SRA distinguishes single-end from paired-end automatically and begins aligning as soon as the first megabases arrive. In practice, alignment finishes within seconds of the completing the download.

Note: If the files already exist in the local `./SRA/` cache directory (from a previous `--sraDownload`), they are used instead of re-downloading.

To evaluate quality-control metrics across several SRA runs quickly, without storing the reads or alignment files:

```
magic2 -x IDX --srr SRR1,SRR2,SRR3 --no_aln --maxGB 1 -o out/
```

5.4 Downloading SRA runs

Use `--sraDownload` to download one or more SRA runs as local files before alignment. The files are placed in the `./SRA/` subdirectory. This may even be useful to prepare data for other bioinformatics pipelines.

Default download format: interleaved FASTA (recommended)

```
magic2 --sraDownload SRR24511885
# Output: SRA/SRR24511885.sample_12.fasta.gz
```

Download as split FASTQ files, limiting to 2 GB:

```
magic2 --sraDownload SRR24511885 --fastq --split_pairs --maxGB 2
# Output: SRA/SRR24511885_R1.fastq.gz
#         SRA/SRR24511885_R2.fastq.gz
```

Processing split files is significantly slower, please prefer the interleaved fasta/fastq formats.

Full syntax:

```
magic2 --sraDownload SRR1,SRR2,... [--fastq] [--split_pairs] [--maxGB <n>]
```

Option	Description
<code>--fastq</code>	Download in FASTQ format (preserves quality scores). Default is FASTA.
<code>--split_pairs</code>	Write paired-end reads to separate <code>_R1</code> and <code>_R2</code> files instead of interleaved (slower, not recommended)
<code>--maxGB <n></code>	Stop after downloading <code>n</code> gigabases. By default the entire run is downloaded.

Single-end runs are automatically exported as single files. The number of SRR identifiers in a single command is not limited.

5.5 Streaming and caching simultaneously

You can align on the fly and cache the reads in one command:

```
magic2 -x IDX.T2T --srr SRR24511885 --fastq --sraCaching -o out/ --full
# Produces: SRA/SRR24511885.sample_12.fastq.gz (cached copy)
#           Full analysis output in out/
```

If the run is already cached, it will not be downloaded again.

5.6 Processing multiple runs in parallel

To process multiple runs in parallel construct a run manifest `-I r.iConfig` as described in (Section 5.2). Data packets from all the runs will be processed in parallel, each run exporting its own results in the global `-o outDir` directory. If all runs are streamed from SRA, you can just list them on the command line `magic2 -srr SRR1234,SRR4567,...` as shown in (Section 10.1).

5.7 Processing multiple sets runs in parallel on a multicore machine

On a large multi-core linux machine (such machine often have 48, or 96 or even more CPUs), Magic selects the least buzy core, and runs just on that one core. The purpose is to eliminate data traffic accross cores, which would significantly affect the performances. But this means that on a 4 core machine, with say 96 CPUs, 3 cores remain idle. As the index is memory-mapped and shared between concurrent Magic2 processes, you may want on a 4 core machine to run 4 sets of runs in parallel using the script:

```
magic2 -x IDX.T2T --srr SRR1234 --full -o out_SRR1234 &
sleep 1 ; magic2 -x IDX.T2T -I r1.iConfig --full -o out_SRR1235 &
sleep 1 ; magic2 -x IDX.T2T -I r1.iConfig --full -o out_SRR1236 &
sleep 1 ; magic2 -x IDX.T2T -I r1.iConfig --full -o out_SRR1237 &
```

Note: Each parallel instance reads the shared index but uses independent output directories. Total RAM usage is the index size plus per-run working memory. Thanks to the 1 second launching delays, each process will automatically select a different (multi-CPU) core.

To know the number of cores and CPUs available on your machine, type the command:

```
ls -lsd /sys/devices/system/node/node*
```

6 Building a Target Index

6.1 Overview

Before aligning any reads you must build an index from your reference genome (and optionally annotation files). The index is written to a directory and reused for all subsequent alignment runs against the same reference.

6.2 The target configuration file (tConfig)

Create a plain-text configuration file with two tab-separated columns: a single-letter type code and a file path. Multiple lines with the same type are allowed and merged. Example:

```
cat > target.tConfig << EOF
G   genome.chromosomes.fasta.gz
M   genome.mitochondria.fasta.gz
R   genome.rRNA.fasta.gz
E   ERCC.fasta.gz
E   DNA_spike_in.fasta
A   genome.annotation.gtf.gz
EOF
```

Type codes:

Code	Label	Description
G	Genome	Genomic sequence. Each chromosome may be on a separate G line.
M	Mitochondria	Mitochondrial sequence. Reported separately because many protocols show high mitochondrial read fractions.
C	Chloroplast	Chloroplast sequence (plant genomes).
R	Ribosomal RNA	rRNA precursor sequences. Strongly recommended: rRNA can constitute up to 80% of a sample and is repeated many times in the genome, causing multi-mapping confusion if not handled explicitly. This fasta file should contain an exemplar of all rRNA precursors.
E	Controls	ERCC spike-in or other sequencing controls.
B	Bacteria	Potential bacterial contamination sequences.
V	Virus	Potential viral contamination sequences.
A	Annotation	Gene annotation in GTF or GFF format, or a <code>.introns</code> file (see below).

Note: Chromosome names in G files must match those used in the A annotation file.

6.2.1 Annotation file formats

Magic2 accepts GTF, GFF, or a custom `.introns` format.

GTF/GFF: Magic2 expects at least 7 columns:

```
chrom  method  tag  a1  a2  .  strand
```

Only `exon` and `CDS` lines are used; all others are ignored. Coordinates are 1-based; `a1 < a2`; strand is `+` or `-`. The chromosome identifiers, column 1 of the gtf/gff files, must match the chromosome identifiers given the the `G` genome fasta files.

.introns format: If no GTF/GFF is available, provide a 3-column introns file:

```
chrom  a1  a2
```

`a1` and `a2` give the positions of the two `G` bases of the GT-AG motif. On the top strand `a1 < a2`; on the bottom strand `a1 > a2`. Coordinates are 1-based.

In introns declared in the gtf/gff or intron file will be listed as known in the results files, the other as novel.

6.3 Creating the index

From a raw genome (FASTA only, for tests):

```
magic2 --createIndex IDX.genome -t genome.fasta.gz
```

From a full configuration file (recommended):

```
magic2 --createIndex IDX.target -T target.tConfig
```

With custom seed parameters:

```
magic2 --createIndex IDX.target -T target.tConfig \  
      --seedLength 18 --step 6
```

Option	Argument	Default	Description
-t	file	—	Single FASTA genome file (raw genome).
-T	file	—	Target configuration file (annotated genome).
--seedLength	int	16 (≤ 200 Mb) or 18 (human)	Seed length in bases. Not recommended to change for metazoan genomes; shorter seeds may help for bacterial projects.
--step	int	6	Seed stepping interval. The best seed among the next step words is selected. Larger values (8–10) reduce RAM and speed up indexing and alignments at some cost to sensitivity.
--maxtargetRepeats	int	81	Seeds repeated more often in the genome are not indexed, effectively masking the highly repetitive regions.

Step is the maximal distance between seed, the average distance is $\text{step}/2$.

7 Running an Alignment

7.1 Basic usage

```
magic2 -x IDX -i reads.fasta.gz -o out/
magic2 -x IDX -i R1.fastq.gz+R2.fastq.gz -o out/
magic2 -x IDX --srr SRR1234,SRR5678 -o out/
```

7.2 Output format options

By default, alignments are written in the custom `.hits` format. You can request other formats with:

```
magic2 -x IDX -i reads.fasta.gz -o out/ --hits --tabular --bam
```

Flag	Description
--hits	Magic <code>.hits</code> format. Describes mismatches, introns, read overhangs, short indels and mismatches explicitly. Default.
--tabular	Magic-BLAST tabular format. Similar to <code>.hits</code>
--sam	Standard SAM format.
--bam	Standard BAM format. Required for tools such as IGV, Samtools, and many genome browsers.
--no_ali	Suppress all alignment output. Useful when only quality metrics or expression counts are needed.

Multiple format flags may be combined; one output file per format is written to the output directory. By default, Magic2 exports in the `.hits` format which is the richest.

7.3 Requesting the full output suite

Adding `--full` produces every available output in a single pass:

```
magic2 -x IDX.T2T --srr SRR1234 -o out/ --full
```

This is equivalent to passing `--genes --wiggles --wiggle_ends` together. See Sections 8.4–8.6 for descriptions of each output type.

8 Output Files

All output files are written to the directory specified with `-o`. The subsections below describe each file type.

8.1 Alignment files

File pattern	Format	Notes
*.hits	Magic hits	Magic1 format, with full mismatches, overhangs and intron detail.
*.tab	Tabular	Magic-BLAST format; easy to parse.
*.bam	BAM	Standard; index with <code>samtools index</code> .

The `.hits` and tabular formats provide more detail than SAM/BAM, including explicit descriptions of mismatches, intron coordinates, and read overhangs. BAM is the appropriate choice when interoperating with third-party tools. Magic2 exports natively in SAM format. SAM is then transformed in BAM by calling automatically `samtools`. If `samtools` is not available on your path, asking for `--bam` will revert to SAM format.

8.2 Intron support table

Magic2 reports all introns observed in the alignment, classified as *known* (present in the I annotation used during index construction) or *novel*.

Output is in the **tag-sample format (TSF)**, a compact tab-separated format in which each line the support of a given intron in a given run. A utility command merges TSF files from multiple runs and produces a human-readable summary table:

```
cat out_dir/*/introns.tsf | bin/tsf -I tsf -O table > human_readable_intron_table.txt
```

Each intron entry records: run, chromosome, donor position, acceptor position, strand, GT-AG motif support, and read count per run. Example:

#Intron	Run	format	n	nR	feet
1__2489274_2489781	RNA_AGLR2_C_1	iit 1	42	gt_ag	
1__2489908_2491261	RNA_AGLR2_C_1	iit 0	13	gt_ag	
1__2491418_2492062	RNA_AGLR2_C_1	iit 0	50	gt_ag	
1__2492154_2492932	RNA_AGLR2_C_1	iit 0	8	gt_ag	
1__2492154_2493111	RNA_AGLR2_C_1	iit 0	37	gt_ag	
1__2492154_2493182	RNA_AGLR2_C_1	iit 0	1	gt_ag	
1__2492154_2493215	RNA_AGLR2_C_1	iit 0	8	gt_ag	

```
1__2493255_2494303 RNA_AGLR2_C_1 iit 0 66 gt_ag
```

8.3 Poly-A addition sites

An addition site is called when at least 8 terminal bases of a read (12 if they contain a non-A/T letter) match a poly-A extension in the 3' end of forward reads or a poly-T extension in the 5' end of reverse reads, and these bases do not align to the genome. This evidence is accumulated per genomic position and reported per run in TSF format, analogous to the intron support table.

```
# PolyA Run ti Strand n
G.1__14497266 RNA_AGLR2_C_1 ti + 1
G.1__14497939 RNA_AGLR2_C_1 ti + 1
G.1__34403194 RNA_AGLR2_C_1 ti + 1
G.1__45401411 RNA_AGLR2_C_1 ti + 1
G.1__103797986 RNA_AGLR2_C_1 ti + 1
G.1__213596326 RNA_AGLR2_C_1 ti + 1
G.1__16266438 RNA_AGLR2_C_1 ti - 1
```

8.4 Gene expression counts

Activated by `--full`, `--wiggles`, or `--genes`.

Magic2 builds a 1-base-resolution coverage map internally during alignment (at no additional I/O cost) and, if annotation was provided, accumulates read counts per gene.

How overlapping genes are resolved: The annotation is projected onto the genome and sliced into non-overlapping regions, each classified as intergenic, intronic, exonic (sense), exonic (anti-sense), or CDS. For reads falling in ambiguous regions (for example, two overlapping genes on opposite strands sharing a 3' UTR), attribution is deferred and distributed between the two genes in proportion to their non-ambiguous exon support.

Strand detection: The strandedness of the library is inferred from the fraction of intron reads exhibiting GT-AG (forward) vs. CT-AC (reverse) motifs. Modern stranded protocols produce values below 1% or above 99%; unstranded protocols fall in the 40–60% range.

Expression index: A preliminary expression index is computed by subtracting the contribution of super-expressed genes (those capturing >1% of all counts), which would otherwise distort comparisons.

Counts are exported in TSF format and can be merged across runs with the same utility used for intron tables. The merged file contains all the counts necessary to study differential expression.

8.5 Coverage plots

Activated by `--wiggles` or `--full`.

Coverage plots are exported in the **BF format** (a UCSC Genome Browser compatible wiggle format) at 10-base resolution by default.

```
magic2 -x IDX -i reads.fasta.gz -o out/ --wiggles
magic2 -x IDX -i reads.fasta.gz -o out/ --wiggles --wiggle_step 1
```

Option	Argument	Default	Description
<code>--wiggles</code>	—	off	Enable coverage plot output.
<code>--wiggle_step</code>	int	10	Resolution in bases. Use 1 for single-base resolution.

BF files can be loaded directly into the UCSC Genome Browser, IGV, or converted to BigWig for large-scale display.

8.6 Transcription start and end sites, repetitions, breakpoints

Activated by `--full` or `--wiggle_ends`.

Magic2 exports special coverage tracks representing the first 20 bases and final 20 bases of fully aligned reads. Because sequencing of an mRNA template is always oriented inward, an accumulation of read endpoints at a particular genomic position indicates a transcription start or termination site. These tracks are useful for refining genome annotations, particularly to extend or correct 5' and 3' UTR boundaries.

Magic2 also export a track of non-unique alignments, showing the expressed repetitive regions. Also a track of incomplete alignments, indicating the candidate break-points.

8.7 Quality-control metrics

Magic2 measures and reports the following metrics for each run:

Metric	Description
Pairs	Number of raw pairs.
Aligned_pairs	Number and fraction of aligned pairs.
Compatible_pairs	Number and fraction of compatible aligned pairs.
Circle_pairs	Number and fraction of locally diverging pairs (candidate circles).
Non_compatible_pairs	Number and fraction of non compatible aligned pairs.
Reads	Total number of raw reads, counting 2 reads per pair if applicable.
Low_entropy_reads	Number of rejected reads with very low sequence complexity.
Low_entropy_bases	Number of bases in those reads.
Aligned_reads	Number of aligned reads.
Unaligned_reads	Number of unaligned reads.
Perfect_reads	Number and fraction of fully and exactly aligned reads.
Complex_reads	Number and fraction of reads aligned in an unusual way.
Partial_reads	Number and fraction of partially aligned reads.
Bases	Number of bases in the raw reads: total, in read 1, in read 2.
Aligned_bases	Number and fraction of aligned bases: total, in read 1, in read 2.
Unaligned_bases	Number of unaligned bases.

Metric	Description
Genome_length	Cumulated length of the calss G (genomic) targets.
CDS_utr_intronic_intergenic_bases	Number and fraction of bases mapping in the different annotated regions.
ATGCN	Number of bases A, T, G, C or other in the raw reads.
Clipped_polyA	Number of reads with at least 8 overhanging terminal A.
Clipped_polyT	Number of reads with at least 8 overhanging initial T.
polyA_sites	Number of observed polyA addition sites.
WiggleCumul	Total number of bases represented in the coverage plots (wiggles).
Errors	Number (and fraction of aligned bases) of mismatched bases.
Mismatches_and_InDels	Number (and fraction of aligned bases) of substitutions and short indels.
Strategy	A positive number in RNA samples, negative in DNA samples.
Intron_supports	Number of local read supports for new and novel introns.
Supported_introns	Number of new and novel supported introns.
Min_read_length	Minimal read length.
Max_read_length	Maximal read length.
Length_distribution_1_5_50_95	Read length distribution per quantile.
Reads_Aligned_once	Number and Fraction of all reads uniquely mapped.
Reads_multi_aligned_n	Number and Fraction of all reads mapped 2,3,4...10 times.
Reads_aligned_several_times	Number and Fraction of all reads mapped several time.
Reads_aligned_in_class_X	Number and Fraction of aligned reads per strand per class (rRNA, Genome...).
Bases_aligned_in_class_X	Number and Fraction of aligned bases per strand per class (rRNA, Genome...).

All metrics are written in the output directory in the human readable TSF-format allowing easy production of tables per property

```
cat out_dir/*/runStats.tsf | gawk -F '\t' '{if($2 == tag)print;} tag=Perfect | bin/
tsf -I tsf -O table --title 'Perfect reads' > perfect_reads.table.txt
```

9 Command-Line Reference

This section lists all recognised options. Options are grouped by function.

9.1 Index construction

Option	Argument	Default	Description
<code>--createIndex</code>	<i>dir</i>	required	Directory to write the index into.
<code>-t</code>	<i>file</i>	—	Single FASTA genome file (no annotation).
<code>-T</code>	<i>file</i>	—	Target configuration file (with annotation).
<code>--seedLength</code>	<i>int</i>	16 or 18	Seed length in bases.
<code>--step</code>	<i>int</i>	6	Seed stepping interval.

9.2 Input

Option	Argument	Default	Description
<code>-x</code>	<i>dir</i>	required	Index directory (from <code>--createIndex</code>).
<code>-i</code>	<i>file</i>	—	Input FASTA/FASTQ file. Use <code>f1+f2</code> for paired-end reads in two files.
<code>--srr</code>	<i>list</i>	—	Comma-separated SRA run identifiers. Streamed from NCBI or read from cache.
<code>--sraDownload</code>	<i>list</i>	—	Download SRA runs to <code>./SRA/</code> and exit.
<code>--fastq</code>	—	off	(With <code>--sraDownload</code>) download in FASTQ format.
<code>--split_pairs</code>	—	off	(With <code>--sraDownload</code>) write pairs to separate <code>_R1/_R2</code> files.
<code>--maxGB</code>	<i>int</i>	unlimited	Limit download or processing to <i>n</i> gigabases.
<code>--sraCaching</code>	—	off	Cache SRA data locally while aligning on the fly.

9.3 Output

Output directory (-o): The full directory path is created automatically if it does not exist, including all intermediate directories (equivalent to `mkdir -p` in Unix). Relative paths are anchored to the current working directory; absolute paths beginning with `/` are used as given.

```
# Relative path: creates ./results/human/run1/ if needed
magic2 -x IDX -i reads.fastq.gz -o results/human/run1/

# Absolute path
magic2 -x IDX -i reads.fastq.gz -o /tmp/magic2_test/
```

Option	Argument	Default	Description
<code>-o</code>	<i>dir</i>	required	Output directory. Created if it does not exist.
<code>--hits</code>	—	on	Export in Magic <code>.hits</code> format.
<code>--tabular</code>	—	off	Export in tabular (Magic-BLAST) format.
<code>--sam</code>	—	off	Export in SAM format.
<code>--bam</code>	—	off	Export in BAM format.
<code>--no_ali</code>	—	off	Suppress alignment output (metrics and counts only).
<code>--full</code>	—	off	Enable all outputs: genes, wiggles, wiggle ends.
<code>--genes</code>	—	off	Export gene expression count tables.
<code>--wiggles</code>	—	off	Export coverage plots (BF/wiggle format).
<code>--wiggle_ends</code>	—	off	Export transcription-start/end coverage tracks.
<code>--wiggle_step</code>	<i>int</i>	10	Coverage plot resolution in bases.

10 Worked Examples

10.1 Quality survey of SRA runs without storing data

You want to assess library quality across six SRA runs before deciding which to analyse further, without consuming disk space.

```
magic2 -x IDX.T2T \
  --srr SRR24511880,SRR24511881,SRR24511882,\
  SRR24511883,SRR24511884,SRR24511885 \
  --no_ali --maxGB 2 -o qc_survey/
```

This streams each run from NCBI, produces quality-control metrics (rRNA fraction, strandedness, adaptor rate, alignment rate), and writes nothing to disk except the metric summary files. No alignment or coverage files are produced.

10.2 Full analysis of a paired-end Illumina run

```
# Build index once (reuse for all subsequent runs)
magic2 --createIndex IDX.hg38 -T hg38.tConfig

# Analyse a local paired-end dataset
magic2 -x IDX.hg38 \
  -i sample_R1.fastq.gz+sample_R2.fastq.gz \
  -o sample_out/ \
  --bam --full
```

```
# Output directory will contain:
#  sample.bam          -- alignments (BAM)
#  introns.tsf         -- known and novel introns with support counts
#  polyA.tsf           -- poly-A addition sites
#  genes.tsf           -- gene expression counts
#  *.BF               -- coverage wiggle tracks
#  tss.tsf / tes.tsf   -- transcription start/end site tracks
#  qc_metrics.tsf      -- quality-control summary
```

10.3 Parallel processing of multiple SRA runs

On a 4-core machine (probably 96 CPUs) you can process four runs simultaneously. The index is memory-mapped and shared; only per-run working memory is duplicated.

```
magic2 -x IDX.T2T --srr SRR100001 --full -o out_SRR100001 &
sleep 1 ; magic2 -x IDX.T2T --srr SRR100002 --full -o out_SRR100002 &
sleep 1 ; magic2 -x IDX.T2T --srr SRR100003 --full -o out_SRR100003 &
sleep 1 ; magic2 -x IDX.T2T --srr SRR100004 --full -o out_SRR100004 &
wait
echo "All runs complete."
```

10.4 Merging multi-run output tables

After analysing several runs, merge the per-run TSF tables into a single comparison table:

```
cat out/*/genes.tsf > all_genes.tsf
bin/tsf -i all_genes.tsf -O table --title 'Gene counts' > all_gene_counts.table.txt
```

11 Troubleshooting

Problem	Solution
magic2: command not found	Ensure the directory containing the <code>magic2</code> binary is on your <code>PATH</code> . Run <code>which magic2</code> to check.
Index creation fails with out-of-memory error	Increase available RAM, or use a larger <code>--step</code> value (e.g. 8 or 10) to reduce index size at a some cost to sensitivity.
Zero rRNA fraction	In RNA-seq experiments, this is unusual. Ensure that an R line pointing to rRNA sequences was included in the tConfig so they are counted correctly.
High rRNA fraction (>50%)	This is a library quality issue, not an error.
Very low alignment rate (<20%)	Check that the correct genome was indexed. For RNA-seq data, verify that the target includes the rRNA and mitochondria files. Possibly consider adding a viral or bacterial contamination fasta file

Problem	Solution
SRA download stalls or times out	Check your network connection to NCBI. Use <code>--maxGB</code> to limit the transfer size, or pre-download with <code>--sraDownload</code> before aligning.
Chromosome names don't match between genome and GTF	This is a fatal error reported during index creation. Ensure that the chromosome identifiers in your FASTA <code>G</code> file exactly match those in the <code>A</code> annotation file (including <code>chr</code> prefix vs. no prefix).
BAM file not produced despite <code>--bam</code> flag	To transform a SAM output into a BAM output, Magic2 invokes 'samtools'. Otherwise Magic2 reverts to SAM format. Samtools is not part of the magic2 distribution. You must install it from the web and make it accessible. Check by typing: <code>which samtools</code> .
Missing gene counts output despite <code>--full</code>	Confirm that the index was built with <code>-T</code> (tConfig including a <code>I</code> annotation line) rather than <code>-t</code> (raw genome only). The actual tConfig file used during index creation is copied in the <code>IDX</code> directory. Please check that file.

Acknowledgements

We thank David Landsman and Tom Madden for their support. This work was supported by the Intramural Research Program of the National Library of Medicine, National Institutes of Health.

References

- [1] D. Thierry-Mieg and J. Thierry-Mieg. AceView: a comprehensive cDNA-supported gene and transcripts annotation. *Genome Biology*, 7(Suppl 1):S12, 2006.
- [2] R. Durbin and J. Thierry-Mieg. The ACEDB genome database. In S. Suhai, editor, *Computational Methods in Genome Research*, pages 45–55. Plenum Press, New York, 1994.
- [3] J. Thierry-Mieg, D. Thierry-Mieg, and G. Boratyn. Magic2 source code and documentation. <https://github.com/NLM-DIR/Magic2>.